

ARDUINO

PROGRAMMER UN COMPORTEMENT
PHOTOTAXIQUE SUR UN ROBOT
MOBILE

21 / 03 / 2025

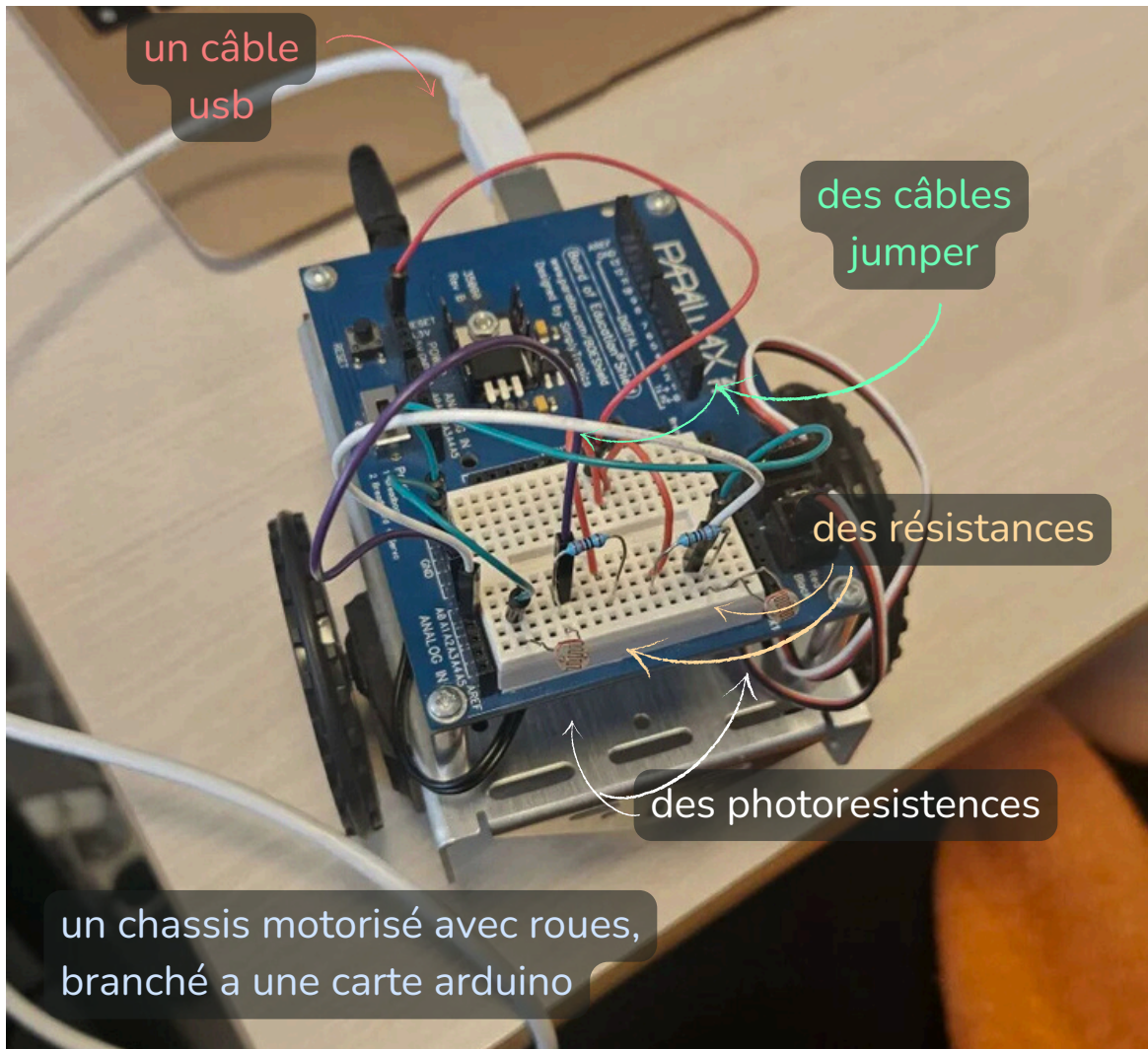
Maiïwenne BEDDAR X Sara BAASSOU

INTRODUCTION

Dans ce projet, nous allons concevoir et programmer un robot mobile capable de réagir à la lumière en se déplaçant vers la source lumineuse la plus intense. Ce comportement, appelé phototaxie, est couramment observé dans le règne animal, notamment chez certains insectes. Pour réaliser cette expérience, nous utiliserons les composants suivants :

- Des câbles jumper pour établir les connexions entre les différents éléments électroniques,
- Des photorésistances (capteurs de lumière) qui permettront au robot de détecter les variations d'intensité lumineuse,
- Des résistances pour assurer le bon fonctionnement des capteurs,
- Un câble USB pour programmer et alimenter la carte,
- Un châssis motorisé avec roues, connecté à une carte Arduino, qui servira de base mobile au robot.

L'objectif est d'écrire un programme qui analyse les données des photorésistances et ajuste la direction des moteurs en fonction de l'intensité lumineuse perçue. Ainsi, le robot pourra naviguer de manière autonome en se dirigeant naturellement vers la source de lumière la plus forte.



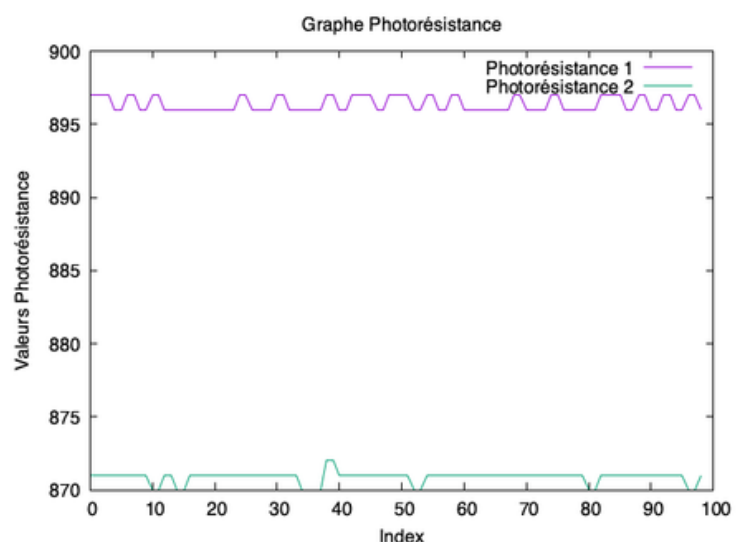
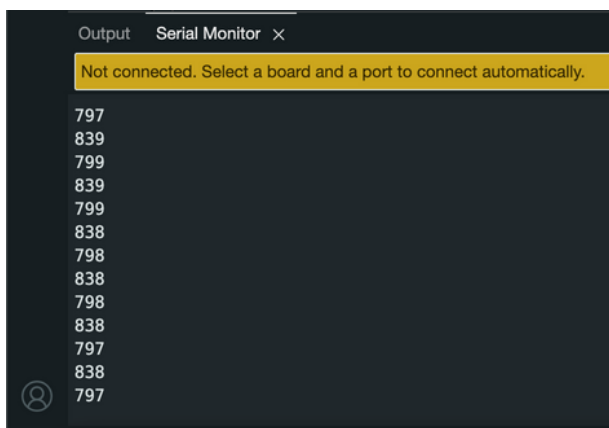
CALIBRAGE DES DONNÉES

Afin de concevoir ce robot mobile capable de réagir à la lumière, nous avons d'abord fait un fichier qui capte la lumière afin de savoir comment ça fonctionne, . Nous avons également ajouté deux fichiers C afin de comparer les valeurs et de pouvoir les calibrer.

Dans un fichier *photoresistance.ino*, nous avons utilisé deux variables *CAPTEUR_DROIT_AVANT* et *CAPTEUR_GAUCHE_AVANT* qui nous ont permis de récupérer les valeurs des deux capteurs de notre robot mobile. Deux variables supplémentaires nous ont servi à afficher ces valeurs : *value* et *value2*. Dans la fonction *setup()*, fonction permettant de faire toutes les initialisation, nous avons initialiser la communication avec le PC avec *Serial.begin(9600)*, et nous avons déclaré les broches A1 et A2 correspondant respectivement aux *CAPTEUR_GAUCHE_AVANT* et *CAPTEUR_DROIT_AVANT* comme *INPUT*, c'est-à-dire, qu'elle reçoit une information (ici l'intensité de la luminosité allant de 0 à 1024 selon qu'il fait très lumineux ou très sombre). Ceci étant fait grâce à la fonction *pinMode()*.

Enfin pour récupérer et afficher ces valeurs nous avons utilisé la fonction *loop()*, qui permet de répéter indéfiniment (tant que le Arduino est allumé) les instructions qu'elle contient. Pour chacune nous avons utilisé *analogRead()* qui permet de lire la valeur de *CAPTEUR_DROIT_AVANT* et de l'assigné à *value* (de meme pour *value2*). et enfin la fonction *Serial.println()* nous a permit d'afficher cette valeur à l'écran (dans le serial).

Notons que Serial est la fenêtre permettant d'afficher ce que notre programme renvoie, elle peut ne pas s'afficher automatiquement, il faut donc aller dans *Tools* et cliquer sur *Serial Monitor*.



Le fichier C, *buffer.c*, nous renvoie deux fichiers, un fichier *monfich.dat* contenant les valeurs captées par la photorésistance 1 et un fichier *monfich2.dat* contenant les valeurs captées par la photorésistance 2 du robot. Nous avons créer un deuxième fichier C *plot.c*, qui nous permet de créer un graphique à partir des valeurs de *monfich.dat* et *monfich2.dat*. Nous avons décider de séparer cette étape en deux fichiers afin de pouvoir modifier les valeurs aberrantes à la main dans les deux fichiers. On remarque que les deux photorésistance, dans un même environnement lumineux plutôt sombre ont un écart de 25 environ (valeur qui nous servira au calibrage plus tard).

EXPLICATIONS DU CODE

Après avoir réalisé le calibrage des photorésistances, nous avons réutilisé cette logique pour programmer ce code. L'objectif est de permettre au robot mobile de se déplacer vers la source lumineuse la plus intense en analysant les données des capteurs et en ajustant la vitesse de ses roues en conséquence. Grâce à cette approche, le robot peut adapter sa trajectoire en fonction de la lumière détectée, lui permettant ainsi de naviguer de manière autonome.

1. Déclaration des constantes et inclusion des bibliothèques

Le programme commence par inclure la bibliothèque Servo.h, qui permet de contrôler les moteurs du robot. Ensuite, les broches associées aux composants sont définies :

- ROUE_GAUCHE (broche 12) et ROUE_DROIT (broche 10) pour les servomoteurs des roues,
- CAPTEUR_GAUCHE (broche A1) et CAPTEUR_DROIT (broche A2) pour les photorésistances.

Deux objets de type Servo, nommés roueGauche et roueDroite, sont ensuite créés pour piloter les moteurs.

2. Initialisation des composants dans setup()

Dans la fonction setup(), les moteurs des roues sont attachés aux broches correspondantes et la communication série est initialisée (Serial.begin(9600)) afin d'afficher les valeurs lues par les capteurs.

3. Lecture des capteurs et prise de décision dans loop()

La boucle principale (loop()) s'exécute en continu et réalise les étapes suivantes :

- Lecture des valeurs des photorésistances (analogRead(CAPTEUR_GAUCHE) et analogRead(CAPTEUR_DROIT)).
- Affichage des valeurs lumineuses sur le moniteur série.
- Comparaison des valeurs pour ajuster la trajectoire du robot :
 - Si la lumière est plus intense à gauche (lumiereGauche > lumiereDroite + 10), le robot tourne à gauche en ralentissant la roue gauche et en maintenant la roue droite à la même vitesse.
 - Si la lumière est plus intense à droite (lumiereDroite > lumiereGauche + 10), le robot tourne à droite en ralentissant la roue droite et en accélérant la roue gauche.
 - Si la lumière est équilibrée, le robot avance tout droit avec des vitesses adaptées aux moteurs (1700 pour la roue gauche et 1300 pour la roue droite).

Un petit délai (delay(100)) est ajouté pour stabiliser les mesures et éviter un trop grand nombre de mises à jour en peu de temps.

4. Optimisation et ajustements

Le seuil de différence de lumière utilisé (+10) permet de s'assurer que les variations minimales ne déclenchent pas de corrections inutiles, tout en tenant compte des éventuelles différences de sensibilité des capteurs. L'envoi des valeurs sur le moniteur série permet également de surveiller les performances du système et d'ajuster les seuils si nécessaire.

Conclusion

Ce programme permet au robot de se déplacer de manière autonome en suivant la lumière grâce à deux photorésistances placées de chaque côté. Il illustre une application concrète de la robotique en combinant la lecture de capteurs et la prise de décision via un microcontrôleur. Une amélioration possible serait d'ajouter des photorésistances supplémentaires à l'avant du robot afin d'augmenter la précision de la détection lumineuse et d'améliorer sa capacité à s'orienter plus efficacement vers la source la plus intense.

ANNEXES

Code permettant de récupérer les valeurs
captées par les photorésistances :

```
#define CAPTEUR_DROIT_AVANT A2
#define CAPTEUR_GAUCHE_AVANT A1

unsigned int value;
unsigned int value2;

void setup() {
    // initialise la communication avec le PC
    Serial.begin(9600);

    // initialise les broches
    pinMode(CAPTEUR_DROIT_AVANT, INPUT);
    pinMode(CAPTEUR_GAUCHE_AVANT, INPUT);
}

void loop() {

    // mesure la tension sur la broche A1
    value = analogRead(CAPTEUR_DROIT_AVANT);
    //Serial.println("value: ");
    Serial.println(value);

    value2 = analogRead(CAPTEUR_GAUCHE_AVANT);
    //Serial.println("value2: ");
    Serial.println(value2);

    delay(500);
}
```

Code C :

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #define __max 200
4
5
6 void affiche_buffer(){
7     int carac;
8     int i = 0;
9     FILE * descripteur;
10    descripteur = fopen("/dev/cu.usbmodem12201", "r");
11
12    FILE * fp;
13    fp = fopen ("monfich.dat", "w+");
14
15    FILE * fp2;
16    fp2 = fopen ("monfich2.dat", "w+");
17
18    while((carac = getc(descripteur)) != EOF && i < __max){
19        i++;
20        fscanf(descripteur, "%d", &carac);
21        printf("valeur : %d \n", carac);
22        if (i%2 == 0){
23            fprintf(fp2, "%d\n", carac);
24            printf("ici i ==: %d\n", i);
25        }
26        else{
27            fprintf(fp, "%d\n", carac);
28            printf("et la i ==: %d\n", i);
29        }
30    }
31    fclose(descripteur);
32    printf("c'est bon!\n");
33 }
34
35 int main(){
36     affiche_buffer();
37     //createGraphs();
38     return 0;
39 }
```

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int createGraphs(){
5     // Open gnuplot using popen
6     FILE *gnuplot = popen("gnuplot -persistent", "w");
7
8     if (gnuplot == NULL) {
9         fprintf(stderr, "Error: Could not open gnuplot.\n");
10        return 1;
11    }
12
13    fprintf(gnuplot, "set terminal pngcairo\n");
14    fprintf(gnuplot, "set output 'graphe.png'\n");
15
16    fprintf(gnuplot, "set title 'Graphe Photorésistance'\n");
17    fprintf(gnuplot, "set xlabel 'Index'\n"); // Or relevant x-axis label
18    fprintf(gnuplot, "set ylabel 'Valeurs Photorésistance'\n"); // y-axis label for sensor values
19    fprintf(gnuplot, "plot 'monfich.dat' with lines title 'Photorésistance 1',
20        'monfich2.dat' with lines title 'Photorésistance 2'\n");
21
22    // Close the gnuplot stream
23    fclose(gnuplot);
24
25    return 0;
26 }
27
28
29 int main(){
30     createGraphs();
31     return 0;
32 }
```

ANNEXES

Code final :

```
1  #include <Servo.h>
2
3  #define ROUE_GAUCHE 12
4  #define ROUE_DROIT 10
5  #define CAPTEUR_GAUCHE A1
6  #define CAPTEUR_DROIT A2
7
8  Servo roueGauche;
9  Servo roueDroite;
10
11 void setup() {
12     Serial.begin(9600);
13     roueDroite.attach(ROUE_DROIT);
14     roueGauche.attach(ROUE_GAUCHE);
15 }
16
17 void loop() {
18     int lumiereGauche = analogRead(CAPTEUR_GAUCHE);
19     int lumiereDroite = analogRead(CAPTEUR_DROIT);
20
21     Serial.println(lumiereGauche);
22     Serial.println(lumiereDroite);
23
24     // Comparaison des capteurs pour ajuster la direction
25     if (lumiereGauche > lumiereDroite + 10) {
26         // Plus de lumière à gauche -> Tourner à GAUCHE
27         roueGauche.writeMicroseconds(1100);
28         roueDroite.writeMicroseconds(1100);
29         Serial.println("Tourne à gauche");
30     } else if (lumiereDroite > lumiereGauche + 10) {
31         // Plus de lumière à droite -> Tourner à DROITE
32         roueGauche.writeMicroseconds(1600);
33         roueDroite.writeMicroseconds(1600);
34         Serial.println("Tourne à droite");
35     } else {
36         // Lumière équilibrée -> Avancer TOUT DROIT
37         roueGauche.writeMicroseconds(1700);
38         roueDroite.writeMicroseconds(1300);
39         Serial.println("Avance tout droit");
40     }
41
42     delay(100);
43 }
```

Sources :

- <https://arduino-france.site/ldr-arduino/>
- <https://fr.vittascience.com/learn/tutorial.php?id=8>
- <https://passionelectronique.fr/photoresistance/>
- <https://arduino.developpez.com/tutoriels/arduino-a-l-ecole/?page=projet-12-utiliser-un-servomoteur>
- <https://arduino-france.site/servo-arduino/>
- <https://docs.arduino.cc/>